

Felipe Dal Col Soares

Destinação Linux

Vila Velha – ES

2026

Felippe Dal Col Soares

Destinação Linux

Projeto apresentado à disciplina de Sistemas Operacionais, do curso de Ciência da Computação da Universidade Vila Velha, como parte da avaliação do primeiro bimestre.

Universidade Vila Velha
Campus de Boa Vista
Curso de Ciência da Computação

Vila Velha – ES
2026

RESUMO

Este trabalho apresenta uma introdução aos sistemas Unix e Linux, tomando como base principal o livro *Introdução ao Linux*, de Arthur Mendes Peixoto e Andreia Gentil Bonfante, publicado pela Escola Superior de Redes em 2013. A organização segue o plano proposto no Projeto do Bimestre da disciplina de Sistemas Operacionais: Unix, GNU, Linux, macOS e Windows, distribuições Linux, terminais e shells, gerenciamento de pacotes e comandos. São abordados ainda conceitos de sistemas de arquivos, permissões, redirecionamento, pipes, Shell Script e ferramentas de administração. Ao final, são apresentados exemplos práticos e um conjunto de atividades com espaço destinado às capturas de tela realizadas no ambiente Linux. **Palavras-chave:** Sistemas Operacionais. Unix. Linux. GNU. Shell. Sistemas de arquivos. Comandos Linux.

SUMÁRIO

	Apresentação	7
	Fonte e delimitação	7
1	UNIX	8
1.1	Conceito de sistema operacional	8
1.2	Arquitetura Unix	8
1.3	Características principais	8
1.4	Histórico	9
1.5	Linha do tempo	9
2	GNU	10
2.1	Software livre e GPL	10
2.2	GPL em relação às licenças tradicionais	10
2.3	GNU e Linux	10
2.4	Nota sobre a terminologia	11
3	LINUX	12
3.1	Origem	12
3.2	Arquitetura e funcionamento	12
3.3	Sistema de arquivos	12
3.4	Permissões	12
3.5	Arquivos e diretórios	13
4	MACOS(X) & WINDOWS	14
4.1	Contextualização	14
4.2	Relação com Unix e Linux	14
4.3	Comparação conceitual	14
5	DISTRIBUIÇÕES DO LINUX	15
5.1	Conceito de distribuição	15
5.2	Principais distribuições apresentadas no livro	15
5.2.1	Red Hat Enterprise Linux	15
5.2.2	CentOS	15
5.2.3	Debian	15
5.2.4	Ubuntu	15
5.2.5	Mandriva	15
5.2.6	Slackware	16

5.3	Escolha da distribuição	16
6	TERMINAIS & SHELLS	17
6.1	Terminal	17
6.2	Shell	17
6.3	Entrada, saída e erro	17
6.4	Pipes	17
6.5	Processos	17
6.6	Shell Script	18
7	GERENCIAMENTO DE PACOTES	19
7.1	Pacotes	19
7.2	RPM	19
7.3	YUM	19
7.4	APT	19
7.5	Comparação	20
8	COMANDOS	21
8.1	Comandos fundamentais	21
8.2	Empacotamento e compactação	21
8.3	Manual de comandos	21
8.4	Atividades práticas exigidas	22
8.4.1	Capítulo 3 – Exercício de fixação 1	22
8.4.2	Capítulo 3 – Atividade 3.2: criando arquivos	22
8.4.3	Capítulo 3 – Atividade 3.3: diretórios e cópia	23
8.4.4	Capítulo 3 – Atividade 3.4: empacotamento e compactação	23
8.4.5	Capítulo 3 – Atividade 3.5: remoção	24
8.4.6	Capítulo 4 – Exercício de fixação 1	24
8.4.7	Capítulo 4 – Atividade 4.1: pesquisando em arquivos	25
8.4.8	Capítulo 4 – Atividade 4.2: contabilizando arquivos	26
8.4.9	Capítulo 4 – Atividade 4.3: controlando a exibição	26
8.4.10	Capítulo 4 – Atividade 4.4: combinando comandos	26
	Considerações finais	28
	Referências	29

LISTA DE ILUSTRAÇÕES

Figura 1 – Linha do tempo simplificada do desenvolvimento Unix/Linux.	9
Figura 2 – Espaço reservado para a resposta do Exercício de Fixação 1 do Capítulo 3.	22
Figura 3 – Espaço reservado para a captura da Atividade 3.2.	22
Figura 4 – Espaço reservado para a captura da Atividade 3.3.	23
Figura 5 – Espaço reservado para a captura da Atividade 3.4.	24
Figura 6 – Espaço reservado para a captura da Atividade 3.5.	24
Figura 7 – Espaço reservado para a captura do exercício do Capítulo 4.	25
Figura 8 – Espaço reservado para a captura da Atividade 4.1.	25
Figura 9 – Espaço reservado para a captura da Atividade 4.2.	26
Figura 10 – Espaço reservado para a captura da Atividade 4.3.	26
Figura 11 – Espaço reservado para a captura da Atividade 4.4.	27

LISTA DE CÓDIGOS

3.1	Exemplo de consulta de permissões	12
3.2	Operações básicas com arquivos e diretórios	13
6.1	Exemplos de redirecionamento	17
6.2	Exemplo de pipeline	17
6.3	Consulta de processos	18
6.4	Exemplo de Shell Script	18
7.1	Operações com RPM	19
7.2	Exemplos de operações com YUM	19
7.3	Exemplos de operações com APT	20
8.1	Empacotamento conforme a atividade do capítulo 3	21
8.2	Consulta ao manual	22
8.3	Exemplo de solução para a Atividade 3.2	22
8.4	Exemplo de solução para a Atividade 3.3	23
8.5	Exemplo de solução para a Atividade 3.4	23
8.6	Exemplo de solução para a Atividade 3.5	24
8.7	Exemplo de uso de pipeline e sort	24
8.8	Exemplo de pesquisa em arquivos	25
8.9	Exemplo de contabilização	26
8.10	Controle da exibição de conteúdo	26
8.11	Modelo de composição de comandos sem sort	26

*

APRESENTAÇÃO

O projeto do bimestre solicita a descrição dos oito primeiros tópicos da disciplina e a utilização de recursos de formatação acadêmica, incluindo resumo, seções, notas de rodapé, equações, linha do tempo, definições, referências cruzadas, figuras, listagens e gráfico. O plano fornecido para a atividade organiza o conteúdo em oito partes: *UNIX*, *GNU*, *Linux*, *MacOS(X) & Windows*, *Distribuições do Linux*, *Terminais & Shells*, *Gerenciamento de pacotes* e *Comandos*.

A principal fonte de conteúdo é o livro *Introdução ao Linux*. O livro apresenta dez capítulos, sendo os oito primeiros dedicados, respectivamente, a histórico e instalação, utilização do sistema, organização do Linux, desvendando o Linux, edição de texto, Shell, Shell Script e estruturas de repetição em Shell Script. Assim, o presente documento reorganiza esses conteúdos de acordo com o plano específico da disciplina.

FONTE E DELIMITAÇÃO

O texto procura preservar a terminologia apresentada no material didático. Quando o plano da disciplina solicita um assunto que não constitui um capítulo específico do livro, ele é apresentado de forma introdutória e explicitamente identificado como complemento necessário ao plano.

1 UNIX

1.1 CONCEITO DE SISTEMA OPERACIONAL

Segundo Peixoto e Bonfante, o sistema operacional atua como interface de comunicação entre o usuário e o hardware. Entre suas funções básicas estão o gerenciamento da CPU, da memória RAM, do armazenamento, dos dispositivos de entrada e saída e a interpretação dos comandos do usuário.

Definição 1 (Sistema operacional) *Camada de software responsável por intermediar a utilização dos recursos de hardware e oferecer serviços para os programas e usuários.*

No caso de sistemas multiusuário, também são relevantes o controle de acesso e a contabilização das atividades do sistema. O livro destaca que diferentes tipos de aplicação podem exigir características especiais, como multitarefa, multiusuário e controle de acesso.

1.2 ARQUITETURA UNIX

O Unix foi projetado com uma arquitetura em camadas. O *kernel* é a parte que depende diretamente do hardware e faz a interface com o restante do sistema. O gerenciador de processos faz parte do kernel e administra os processos em execução e a divisão do tempo de processamento da CPU.

O *shell*, por sua vez, não pertence ao kernel. Ele funciona como interpretador de comandos e também oferece recursos que permitem automatizar tarefas.

1.3 CARACTERÍSTICAS PRINCIPAIS

Entre as características apresentadas no livro estão:

- portabilidade;
- capacidade multiusuário;
- multiprocessamento;
- estrutura hierárquica de diretórios;
- interpretador de comandos (*shell*);
- uso de *pipelines*;

- conjunto de utilitários;
- suporte ao desenvolvimento de software;
- maturidade e confiabilidade.

O uso de *pipelines* permite conectar a saída de um comando à entrada de outro. Esse mecanismo possibilita combinar ferramentas existentes para produzir operações mais complexas.

1.4 HISTÓRICO

O Unix surgiu no Bell Laboratories em 1969, a partir do trabalho de Ken Thompson. O desenvolvimento buscava independência em relação ao hardware. A linguagem B evoluiu posteriormente para a linguagem C, associada ao trabalho de Dennis Ritchie, permitindo reescrever o Unix e facilitar sua portabilidade.

A história apresentada no livro inclui versões como Unix Sixth Edition, PWB Unix, Unix Version 7, System III, System V e Berkeley Unix (BSD).

1.5 LINHA DO TEMPO

A Figura 1 sintetiza alguns marcos mencionados no material e relacionados ao desenvolvimento posterior do ecossistema Unix/Linux.

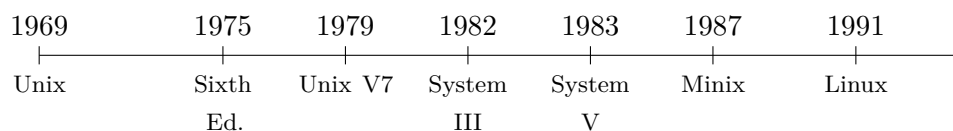


Figura 1 – Linha do tempo simplificada do desenvolvimento Unix/Linux.

2 GNU

2.1 SOFTWARE LIVRE E GPL

O livro apresenta o projeto GNU e a Free Software Foundation (FSF) como elementos importantes para o desenvolvimento dos atuais utilitários e aplicativos associados ao Linux. A GPL aparece como uma licença que procura garantir a liberdade de compartilhar e alterar software livre, mantendo o acesso ao código-fonte.

O conceito de *copyleft* é apresentado como uma utilização do instrumento jurídico do copyright com termos de licenciamento destinados a preservar essas liberdades.

Definição 2 (GPL) *Licença de software livre que, conforme apresentado no material, garante direitos de uso, modificação e redistribuição do software e de trabalhos derivados sob as condições da licença.*

2.2 GPL EM RELAÇÃO ÀS LICENÇAS TRADICIONAIS

O exercício de fixação do Capítulo 1 solicita a comparação entre as licenças Unix e GPL considerando usuários, desenvolvedores, fabricantes de hardware e distribuidores. Em termos do conteúdo do livro, a principal diferença apresentada está no acesso ao código e nas liberdades concedidas ao usuário.

Tabela 1 – Síntese da comparação apresentada no material

Aspecto	GPL / software livre
Código-fonte	Acesso permitido, de acordo com as condições da licença.
Uso	O software pode ser utilizado pelos usuários.
Modificação	A licença permite alteração do software nos termos estabelecidos.
Redistribuição	É permitida conforme as condições da GPL.
Derivados	As condições de licenciamento procuram preservar as liberdades do software livre.

2.3 GNU E LINUX

A relação entre GNU e Linux pode ser entendida como uma combinação entre um kernel Linux e diversos utilitários e componentes de software livre. O livro destaca que o projeto GNU e os trabalhos da Universidade de Berkeley foram fontes importantes para os utilitários e aplicativos presentes no ecossistema Linux.

2.4 NOTA SOBRE A TERMINOLOGIA

O termo “Linux” é utilizado no trabalho seguindo a terminologia do livro, que apresenta as distribuições Linux como mecanismos de fornecimento de software que incorporam o kernel, utilitários, interfaces e aplicações.

3 LINUX

3.1 ORIGEM

O livro apresenta Linus Torvalds como o estudante que, a partir do contato com o Minix, decidiu desenvolver seu próprio sistema. Os primeiros kernels foram disponibilizados na Internet, permitindo que outras pessoas contribuíssem para o desenvolvimento.

A abertura do código possibilitou a participação de estudantes, profissionais, curiosos e outros colaboradores. O material destaca ainda a influência do GNU e do BSD no desenvolvimento dos utilitários e aplicativos do Linux.

3.2 ARQUITETURA E FUNCIONAMENTO

O Linux mantém conceitos característicos de sistemas Unix: organização hierárquica do sistema de arquivos, processos, usuários, permissões, shell e utilitários que podem ser combinados.

$$T_{\text{CPU}} = \sum_{i=1}^n t_i \quad (3.1)$$

A Equação 3.1 representa, de maneira simplificada, a soma dos intervalos de tempo de CPU atribuídos a um conjunto de processos. Ela serve aqui apenas como representação matemática do compartilhamento do recurso de processamento, e não como fórmula específica de escalonamento do Linux.

3.3 SISTEMA DE ARQUIVOS

O Capítulo 3 do livro apresenta o sistema de arquivos do Linux, incluindo *inodes*, tipos de arquivos, diretórios, arquivos de dispositivos, *named pipes*, *sockets*, links, atributos e permissões.

A estrutura hierárquica possui a raiz representada por “/”. A navegação pode ser realizada com o comando `cd`, enquanto `pwd` apresenta o diretório corrente.

3.4 PERMISSÕES

Os arquivos possuem permissões associadas ao proprietário, ao grupo e aos demais usuários. As permissões básicas são leitura (**r**), escrita (**w**) e execução (**x**).

```
1 $ ls -l
```

```
2 $ chmod 644 arquivo.txt
3 $ chmod 755 script.sh
```

Código 3.1 – Exemplo de consulta de permissões

3.5 ARQUIVOS E DIRETÓRIOS

Entre as operações apresentadas no livro estão criação, cópia, remoção, movimentação, listagem, busca e compactação.

```
1 $ touch exemplo1
2 $ mkdir -p ~/temp/exemplos
3 $ cp exemplo1 ~/temp/exemplos/
4 $ mv exemplo1 exemplo2
5 $ rm exemplo2
6 $ ls -l
7 $ pwd
```

Código 3.2 – Operações básicas com arquivos e diretórios

4 MACOS(X) & WINDOWS

4.1 CONTEXTUALIZAÇÃO

O livro utilizado na disciplina concentra-se em Unix/Linux e não possui um capítulo específico dedicado ao macOS e ao Windows. Assim, esta seção é uma comparação introdutória necessária para atender ao plano do projeto, sem atribuir ao livro uma abordagem que ele não apresenta diretamente.

4.2 RELAÇÃO COM UNIX E LINUX

O Unix é apresentado no livro como uma família de sistemas com arquitetura em camadas, shell, processos e sistema hierárquico de arquivos. Sistemas posteriores podem compartilhar conceitos semelhantes, ainda que tenham interfaces e implementações diferentes.

No contexto deste projeto, o ponto central da comparação é que sistemas operacionais diferentes oferecem mecanismos para gerenciamento de processos, arquivos, memória, dispositivos e interação com o usuário, mas podem utilizar ferramentas e interfaces diferentes.

4.3 COMPARAÇÃO CONCEITUAL

Tabela 2 – Comparação conceitual para o projeto

Aspecto	Unix/Linux	Windows/macOS
Interface	Pode utilizar ambiente gráfico e linha de comando.	Também oferece ambiente gráfico e ferramentas de linha de comando.
Arquivos	Organização hierárquica e permissões.	Também possui organização hierárquica e mecanismos próprios de controle de acesso.
Processos	Gerenciados pelo sistema operacional.	Gerenciados pelo sistema operacional.
Terminal	Shell e utilitários Unix.	Ferramentas de terminal próprias e/ou compatíveis com ambientes Unix.
Objetivo do projeto	Administração e uso do sistema Linux.	Comparação conceitual com outros sistemas.

5 DISTRIBUIÇÕES DO LINUX

5.1 CONCEITO DE DISTRIBUIÇÃO

Uma distribuição reúne o kernel Linux com utilitários, interfaces gráficas, aplicações, documentação e mecanismos de instalação e administração. O livro apresenta diferentes distribuições e destaca que a escolha depende do uso pretendido.

5.2 PRINCIPAIS DISTRIBUIÇÕES APRESENTADAS NO LIVRO

5.2.1 Red Hat Enterprise Linux

O material destaca interfaces gráficas, versões voltadas para cliente e servidor e o uso dos gerenciadores RPM e YUM.

5.2.2 CentOS

O livro descreve o CentOS como derivado do Red Hat Enterprise Linux e destaca sua utilização em servidores, o uso de RPM e YUM e a disponibilidade de ferramentas administrativas.

5.2.3 Debian

O Debian é apresentado como uma distribuição desenvolvida por uma comunidade diversificada de programadores, utilizando pacotes DEB e ferramentas como APT e Aptitude.

5.2.4 Ubuntu

O Ubuntu aparece entre as principais distribuições abordadas no capítulo de histórico e instalação.

5.2.5 Mandriva

O material apresenta o Mandriva como uma distribuição voltada para desktops, com opções de idioma e ferramentas de configuração, incluindo ferramentas para mouse, disco e conexões de rede.

5.2.6 Slackware

O Slackware é descrito com destaque para desempenho e estabilidade, além da possibilidade de utilização em servidores. O livro também observa que sua instalação é mais complexa.

5.3 ESCOLHA DA DISTRIBUIÇÃO

O exercício do livro propõe observar o enfoque de cada distribuição, como uso em servidores ou estações de trabalho, suporte comercial, idiomas, plataformas e formas de obtenção.

Tabela 3 – Características das distribuições conforme o material didático

Distribuição	Características destacadas no livro
Red Hat Enterprise Linux	Cliente/servidor, KDE, RPM e YUM, ferramentas de configuração.
CentOS	Derivada do RHEL, servidores, RPM e YUM, comunidade e suporte.
Debian	Pacotes DEB, APT e Aptitude, comunidade diversificada.
Ubuntu	Distribuição apresentada entre as principais distribuições Linux.
Mandriva	Desktop, múltiplos idiomas e ferramentas de configuração.
Slackware	Desempenho, estabilidade, servidores e instalação mais complexa.

6 TERMINAIS & SHELLS

6.1 TERMINAL

O terminal é uma interface pela qual o usuário pode executar comandos. O Capítulo 2 apresenta, por exemplo, o xterm como emulador de terminal do GNOME e o Konsole como simulador de terminal do KDE.

6.2 SHELL

O shell é o interpretador de comandos. Ele recebe comandos, interpreta as solicitações do usuário e encaminha as operações ao sistema. Além disso, possui recursos de programação e automação.

6.3 ENTRADA, SAÍDA E ERRO

O Capítulo 4 apresenta a entrada padrão, a saída padrão e a saída padrão de erros. O redirecionamento permite direcionar essas informações para arquivos.

```
1 $ comando > saida.txt
2 $ comando >> saida.txt
3 $ comando < entrada.txt
4 $ comando 2> erro.txt
```

Código 6.1 – Exemplos de redirecionamento

6.4 PIPES

O operador | conecta a saída padrão de um comando à entrada padrão do comando seguinte.

```
1 $ ps | grep bash
2 $ ls -l | wc -l
```

Código 6.2 – Exemplo de pipeline

6.5 PROCESSOS

O Capítulo 6 aborda gerenciamento, criação, execução em *background*, *daemons*, sinais e visualização de processos.

```
1 $ ps
2 $ ps aux
3 $ top
4 $ jobs
5 $ comando &
```

Código 6.3 – Consulta de processos

6.6 SHELL SCRIPT

Shell Script permite transformar uma sequência de comandos em um programa executável. O livro apresenta variáveis, testes, leitura de dados, parâmetros de linha de comando, estruturas de decisão, expressões regulares, repetições, funções e arrays.

```
1 #!/bin/bash
2
3 nome="$1"
4
5 if [ -z "$nome" ]; then
6     echo "Informe um nome."
7     exit 1
8 fi
9
10 echo "Olá, $nome!"
```

Código 6.4 – Exemplo de Shell Script

7 GERENCIAMENTO DE PACOTES

7.1 PACOTES

Embora o gerenciamento de pacotes seja tratado diretamente no Capítulo 9 do livro, ele faz parte do plano do projeto. Um pacote é descrito como um arquivo que contém uma aplicação ou biblioteca, scripts de instalação e informações como descrição e pré-requisitos.

Os dois formatos destacados no material são RPM e DEB.

7.2 RPM

O RPM foi criado originalmente pela Red Hat e é utilizado por distribuições como Red Hat e Fedora. O comando `rpm` permite instalar, remover, consultar e atualizar pacotes.

```
1 # rpm -ivh pacote.rpm
2 # rpm -e pacote
3 # rpm -Uvh pacote.rpm
4 # rpm -q pacote
```

Código 7.1 – Operações com RPM

7.3 YUM

O YUM trabalha com repositórios e automatiza tarefas relacionadas à obtenção e instalação de pacotes. O livro descreve repositórios dos tipos *base*, *updates* e *extras*.

```
1 # yum search pacote
2 # yum install pacote
3 # yum update
4 # yum remove pacote
5 # yum provides arquivo
```

Código 7.2 – Exemplos de operações com YUM

7.4 APT

O APT é apresentado como ferramenta de gerenciamento de pacotes utilizada nas distribuições Debian e suas variantes. Ele trabalha com dependências e mantém informações sobre os pacotes disponíveis e instalados.

```
1 # apt-get update
2 # apt-get install pacote
3 # apt-get remove pacote
```

Código 7.3 – Exemplos de operações com APT

7.5 COMPARAÇÃO

Tabela 4 – Gerenciadores de pacotes

Ferramenta	Ecossistema	Função
rpm	RPM	Instalação, remoção, consulta e atualização de pacotes.
YUM	Red Hat/CentOS/Fedora no contexto do livro	Consulta repositórios e automatiza instalação e atualização.
APT	Debian e derivados	Gerencia pacotes e dependências.

8 COMANDOS

8.1 COMANDOS FUNDAMENTAIS

Os capítulos 3 e 4 do livro apresentam comandos para navegar, criar, manipular, procurar e analisar arquivos.

Tabela 5 – Comandos fundamentais utilizados no projeto

Comando	Finalidade
<code>pwd</code>	Mostra o diretório corrente.
<code>cd</code>	Altera o diretório corrente.
<code>ls</code>	Lista arquivos e diretórios.
<code>touch</code>	Cria arquivo vazio ou atualiza sua marca temporal.
<code>mkdir</code>	Cria diretórios.
<code>cp</code>	Copia arquivos e diretórios.
<code>mv</code>	Move ou renomeia arquivos e diretórios.
<code>rm</code>	Remove arquivos e diretórios.
<code>find</code>	Procura arquivos e diretórios.
<code>grep</code>	Pesquisa padrões no conteúdo.
<code>wc</code>	Contabiliza linhas, palavras e caracteres.
<code>head</code>	Exibe o início de um arquivo.
<code>tail</code>	Exibe o final de um arquivo.
<code>cut</code>	Seleciona trechos de linhas.
<code>diff</code>	Compara arquivos.
<code>tar</code>	Empacota e desempacota arquivos.
<code>man</code>	Consulta páginas de manual.

8.2 EMPACOTAMENTO E COMPACTAÇÃO

O livro explica que o comando `tar` pode reunir diversos arquivos em um único arquivo, sendo útil para cópias de segurança, transporte e distribuição. A compactação pode ser combinada ao empacotamento.

```
1 $ tar -czvf exemplos.tar.gz exemplo1 exemplo2
2 $ tar -xzvf exemplos.tar.gz
```

Código 8.1 – Empacotamento conforme a atividade do capítulo 3

8.3 MANUAL DE COMANDOS

A documentação local pode ser consultada com `man`.

```
1 $ man ls
2 $ man tar
3 $ man grep
```

Código 8.2 – Consulta ao manual

8.4 ATIVIDADES PRÁTICAS EXIGIDAS

O projeto determina atividades dos capítulos 3 e 4. As subseções seguintes estão preparadas para receber as respostas e as capturas de tela do terminal.

8.4.1 Capítulo 3 – Exercício de fixação 1

O exercício trata do conhecimento da estrutura do sistema de arquivos. A resposta deve ser apresentada juntamente com a captura do terminal solicitada pelo professor.



```
root@localhost:~# tree -L 3 -d /
```

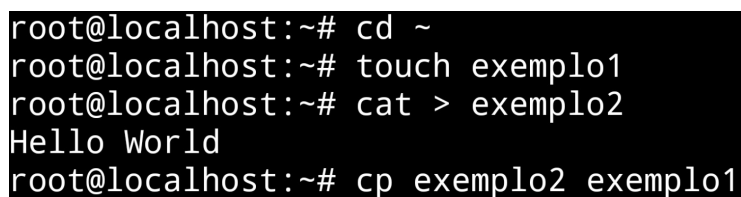
Figura 2 – Espaço reservado para a resposta do Exercício de Fixação 1 do Capítulo 3.

8.4.2 Capítulo 3 – Atividade 3.2: criando arquivos

A atividade solicita a criação dos arquivos `exemplo1` e `exemplo2` usando comandos diferentes, a entrada de algumas linhas em um dos arquivos e a reprodução do conteúdo no arquivo inicialmente vazio.

```
1 $ cd ~
2 $ touch exemplo1
3 $ cat > exemplo2
4 $ cp exemplo2 exemplo1
```

Código 8.3 – Exemplo de solução para a Atividade 3.2



```
root@localhost:~# cd ~
root@localhost:~# touch exemplo1
root@localhost:~# cat > exemplo2
Hello World
root@localhost:~# cp exemplo2 exemplo1
```

Figura 3 – Espaço reservado para a captura da Atividade 3.2.

8.4.3 Capítulo 3 – Atividade 3.3: diretórios e cópia

A atividade solicita a criação do diretório `temp/exemplos` em `home`, utilizando apenas um comando de criação, seguida da movimentação dos arquivos e manutenção de uma cópia do segundo arquivo na origem.

```
1 $ mkdir -p ~/temp/exemplos
2 $ mv exemplo1 exemplo2 ~/temp/exemplos/ && cp exemplo2 ~/temp/
  exemplos/
3 $ cp ~/temp/exemplos
```

Código 8.4 – Exemplo de solução para a Atividade 3.3

```
root@localhost:~# mkdir -p ~/temp/exemplos
root@localhost:~# mv exemplo1 ~/temp/exemplos/ && cp exemplo2 ~/temp/exemplos/
root@localhost:~# cd ~/temp/exemplos
root@localhost:~/temp/exemplos# tar -czvf exemplos.tar.gz exemplo1 exemplo2
exemplo1
exemplo2
root@localhost:~/temp/exemplos# mkdir ~/temp/exemplos2
root@localhost:~/temp/exemplos# mv exemplos.tar.gz ~/temp/exemplos2/
root@localhost:~/temp/exemplos# cd ~/temp/exemplos2
root@localhost:~/temp/exemplos2# tar -xzvf exemplos.tar.gz
exemplo1
exemplo2
```

Figura 4 – Espaço reservado para a captura da Atividade 3.3.

8.4.4 Capítulo 3 – Atividade 3.4: empacotamento e compactação

A atividade solicita a criação de `exemplos.tar.gz`, a criação do diretório `exemplos2`, a movimentação do arquivo e, posteriormente, sua descompactação e desempacotamento.

```
1 $ cd ~/temp/exemplos
2 $ tar -czvf exemplos.tar.gz exemplo1 exemplo2
3 $ mkdir ../exemplos2
4 $ mv exemplos.tar.gz ../exemplos2/
5 $ cd ../exemplos2
6 $ tar -xzvf exemplos.tar.gz
```

Código 8.5 – Exemplo de solução para a Atividade 3.4

```
root@localhost:~# cd ~/temp/exemplos
root@localhost:~/temp/exemplos# tar -czvf exemplos.tar.gz exemplo1 exemplo2
exemplo1
exemplo2
root@localhost:~/temp/exemplos# mkdir ~/temp/exemplos2
mkdir: /root/temp/exemplos2: File exists
root@localhost:~/temp/exemplos# mv exemplos.tar.gz ~/temp/exemplos2/
root@localhost:~/temp/exemplos# cd ~/temp/exemplos2
root@localhost:~/temp/exemplos2# tar -xzvf exemplos.tar.gz
exemplo1
exemplo2
```

Figura 5 – Espaço reservado para a captura da Atividade 3.4.

8.4.5 Capítulo 3 – Atividade 3.5: remoção

Ao final das atividades, a situação original deve ser restaurada, removendo os elementos criados.

```
1 $ rm -f ~/exemplo1 ~/exemplo2
2 $ rm -rf ~/temp
```

Código 8.6 – Exemplo de solução para a Atividade 3.5

```
root@localhost:~/temp/exemplos2# cd ~
root@localhost:~# rm exemplo2
root@localhost:~# rm -rf ~/temp
root@localhost:~# cd ~/temp/exemplos2
bash: cd: /root/temp/exemplos2: No such file or directory
```

Figura 6 – Espaço reservado para a captura da Atividade 3.5.

8.4.6 Capítulo 4 – Exercício de fixação 1

O exercício trabalha o comando `sort`. O capítulo apresenta o *pipeline* como mecanismo para combinar comandos.

```
1 $ echo -e "Felipe 19\nCarlos 21\nBeatriz 20" > ficheiro.txt
2 $ sort -k 2 ficheiro.txt
```

Código 8.7 – Exemplo de uso de pipeline e sort

```

root@localhost:~# cat > questao1
O comando sort é utilizado no Linux para ordenar as linhas de ficheiros de texto
(ou dados provenientes da entrada padrão) em ordem alfabética ou numérica, exibindo
o resultado ordenado no terminal.
root@localhost:~# echo -e "Felippe 19\nCarlos 21\nBeatriz 20" > ficheiro.txt
root@localhost:~# sort -k 2 ficheiro.txt
Felippe 19
Beatriz 20
Carlos 21
root@localhost:~# sort -r 2 ficheiro.txt
sort: cannot read: 2: No such file or directory
root@localhost:~# sort -r ficheiro.txt
Felippe 19
Carlos 21
Beatriz 20
root@localhost:~# sort -n ficheiro.txt
Beatriz 20
Carlos 21
Felippe 19
root@localhost:~# sort -u ficheiro.txt
Beatriz 20
Carlos 21
Felippe 19

```

Figura 7 – Espaço reservado para a captura do exercício do Capítulo 4.

8.4.7 Capítulo 4 – Atividade 4.1: pesquisando em arquivos

A atividade utiliza ferramentas de pesquisa apresentadas no capítulo.

```

1 $ grep -vn "Unix" atividade | tee atividade_temp

```

Código 8.8 – Exemplo de pesquisa em arquivos

```

root@localhost:~# cat << 'EOF' > atividade
> abcdefghijklmUnixnopqrstuvwxyz
> abcdefghUnixjklmnopqrstuvwxyzUnix
> abcdefghijklmnopqrstuvwxyzLinux
> abcdefghijklmnopqrstuUnixvwxyz
> Unixabcdefghijklmnopqrstuvwxyz
> abcdefghijklmnopqrstuvwxyzUnix
> abcdefghijklmnopLinuxqrstuvwxyz
> abcdefhUnixijklmnopqrstuUnixvwxyz
> EOF
root@localhost:~#
root@localhost:~# grep -vn "Unix" atividade | tee atividade_temp
3:abcdefghijklmnopqrstuvwxyzLinux
7:abcdefghijklmnopLinuxqrstuvwxyz

```

Figura 8 – Espaço reservado para a captura da Atividade 4.1.

8.4.8 Capítulo 4 – Atividade 4.2: contabilizando arquivos

O comando `grep` Filtrar linhas que contêm um padrão específico.

```
1 $ grep -c "Unix" atividade
```

Código 8.9 – Exemplo de contabilização

```
root@localhost:~# grep -c "Unix" atividade
6
```

Figura 9 – Espaço reservado para a captura da Atividade 4.2.

8.4.9 Capítulo 4 – Atividade 4.3: controlando a exibição

Para controlar a quantidade de conteúdo exibido podem ser utilizados comandos como `head`, `tail`, `cut` e outros recursos apresentados no capítulo.

```
1 $ sed -n '3,4p' atividade
```

Código 8.10 – Controle da exibição de conteúdo

```
root@localhost:~# sed -n '3,4p' atividade
abcdefghijklmnopqrstvwxyzLinux
abcdefghijklmnopqrstuUnixvwxyz
```

Figura 10 – Espaço reservado para a captura da Atividade 4.3.

8.4.10 Capítulo 4 – Atividade 4.4: combinando comandos

A atividade 4.4 exige a combinação de comandos do capítulo e especifica que não devem ser usados os comandos `sort`, `cat` e/ou `tac`. A solução final deve ser ajustada ao arquivo e à condição exata fornecidos no roteiro utilizado pelo grupo.

```
1 $ grep "padrao" arquivo.txt | wc -l
2 $ head -n 10 arquivo.txt | grep "padrao"
3 $ tail -n 10 arquivo.txt | grep "padrao"
```

Código 8.11 – Modelo de composição de comandos sem sort

```
root@localhost:~# sort -n clientes
12341 Aluno1 Rua Goiás, 25 Planaltina DF
12342A Auno2A v. Beira Rio, 1.005 Vtória ES
12343 Aluno3 Rua Guarani Campinas SP
12344A Aluno4 SQS 2005F 306B Brasília DF
12345 Aluno55 Av Atlântica, 110 Rio de Janeiro RJ
12346 Aluno6 Av. Castro Alves, 320 Salvador BA
12347 Aluno7 Rua da Praia, 215 Porto Alegre RS
12348 Aluno8 R. das Laranjeiras, 1.020 Rio de Janeiro RJ
root@localhost:~# head -n 10 clientes
12348 Aluno8 R. das Laranjeiras, 1.020 Rio de Janeiro RJ
12347 Aluno7 Rua da Praia, 215 Porto Alegre RS
12346 Aluno6 Av. Castro Alves, 320 Salvador BA
12345 Aluno55 Av Atlântica, 110 Rio de Janeiro RJ
12344A Aluno4 SQS 2005F 306B Brasília DF
12343 Aluno3 Rua Guarani Campinas SP
12342A Auno2A v. Beira Rio, 1.005 Vtória ES
12341 Aluno1 Rua Goiás, 25 Planaltina DF
root@localhost:~# tail -n 10 clientes
12348 Aluno8 R. das Laranjeiras, 1.020 Rio de Janeiro RJ
12347 Aluno7 Rua da Praia, 215 Porto Alegre RS
12346 Aluno6 Av. Castro Alves, 320 Salvador BA
12345 Aluno55 Av Atlântica, 110 Rio de Janeiro RJ
12344A Aluno4 SQS 2005F 306B Brasília DF
12343 Aluno3 Rua Guarani Campinas SP
12342A Auno2A v. Beira Rio, 1.005 Vtória ES
12341 Aluno1 Rua Goiás, 25 Planaltina DF
```

Figura 11 – Espaço reservado para a captura da Atividade 4.4.

CONSIDERAÇÕES FINAIS

O estudo dos oito tópicos permite relacionar a história do Unix ao desenvolvimento do Linux e aos conceitos práticos de administração do sistema. O material utilizado destaca a importância do kernel, do shell, da estrutura hierárquica de arquivos, das permissões, dos processos e da combinação de comandos por meio de pipelines.

As atividades práticas dos capítulos 3 e 4 complementam a parte conceitual, pois exigem a criação e manipulação de arquivos, diretórios, pacotes compactados, pesquisa e processamento de informações no terminal. Os capítulos 6, 7 e 8 ampliam essa base ao apresentar processos, Shell Script, estruturas de decisão, repetição, funções e arrays.

Dessa forma, o conteúdo estudado conecta conceitos de sistemas operacionais com tarefas concretas de utilização e administração de um ambiente Linux.

REFERÊNCIAS

PEIXOTO, Arthur Mendes; BONFANTE, Andreia Gentil. **Introdução ao Linux**. Rio de Janeiro: Escola Superior de Redes, 2013. ISBN 978-85-63630-19-3.

UNIVERSIDADE VILA VELHA. **Projeto do Bimestre: Destinação Linux**. Sistemas Operacionais. Ciência da Computação. Campus de Boa Vista, 2026/2.